

CSCI-B 651 NLP Spring 2026 Homework 2

February 5 2026

Instructions

We will be using Canvas to collect your assignments. Please read the following instructions to prepare your submission.

1. Submit your solution in a pdf file and a zip file (<yourLastName_FirstName>.pdf/zip). Your write-up must be in pdf. Your code must be in the zip file.
2. In your pdf file, the solution to each problem should start on a new page.
3. Latex is strongly encouraged to write your solutions, e.g., using Overleaf (<https://www.overleaf.com/>). However, scanned handwritten copies are also acceptable. Hard copies will not be accepted.
4. You may discuss the problems and potential directions for solving them with another student. However, you need to write your own solutions and code separately, and not as a group activity. Please list the students you collaborated with on your submission.

Problem 1 (10 points)

We are given the following text data. Each row represents a document which consists of two words

Word 1	Word 2	Label
old	burger	on-sale
old	pickle	on-sale
tangy	juice	on-sale
tangy	juice	on-sale
tangy	juice	sold-out
tangy	pickle	sold-out
delicious	pickle	on-sale
delicious	burger	sold-out
delicious	juice	sold-out

Table 1: Text data for Problem 1

- (a) Compute all the probabilities required to build a Naive Bayesian classifier. Ignore smoothing. Show your work.

$$\begin{aligned}
 P(\text{old} \mid \text{on-sale}) &= \frac{2}{10} & P(\text{old} \mid \text{sold-out}) &= \frac{0}{8} \\
 P(\text{tangy} \mid \text{on-sale}) &= \frac{2}{10} & P(\text{tangy} \mid \text{sold-out}) &= \frac{2}{8} \\
 P(\text{delicious} \mid \text{on-sale}) &= \frac{1}{10} & P(\text{delicious} \mid \text{sold-out}) &= \frac{2}{8} \\
 P(\text{burger} \mid \text{on-sale}) &= \frac{1}{10} & P(\text{burger} \mid \text{sold-out}) &= \frac{1}{8} \\
 P(\text{pickle} \mid \text{on-sale}) &= \frac{2}{10} & P(\text{pickle} \mid \text{sold-out}) &= \frac{1}{8} \\
 P(\text{juice} \mid \text{on-sale}) &= \frac{2}{10} & P(\text{juice} \mid \text{sold-out}) &= \frac{2}{8}
 \end{aligned}$$

- (b) Using the probabilities from (a), compute the label for a new document “tangy burger”. Show your work.

$$\begin{aligned}
 P(\text{class} \mid \text{data}) &= P(\text{class}) \times P(\text{word} \mid \text{class}) \\
 P(\text{on-sale} \mid \text{tangy burger}) &= P(\text{on-sale}) \times P(\text{tangy} \mid \text{on-sale}) \times P(\text{burger} \mid \text{on-sale}) \\
 &= \frac{5}{9} \times \frac{2}{10} \times \frac{1}{10} = \frac{1}{90} \\
 P(\text{sold-out} \mid \text{tangy burger}) &= P(\text{sold-out}) \times P(\text{tangy} \mid \text{sold-out}) \times P(\text{burger} \mid \text{sold-out}) \\
 &= \frac{4}{9} \times \frac{2}{8} \times \frac{1}{8} = \frac{1}{72}
 \end{aligned}$$

$P(\text{sold-out} \mid \text{tangy burger}) > P(\text{on-sale} \mid \text{tangy burger})$
 Therefore, the predicted label for “tangy burger” is: sold-out

Problem 2 (10 points)

Use the Penn Treebank tagset to tag each word in the following sentences. You may ignore punctuation.

- (a) I take steps forward every day, no matter how small they may be.
I(PRP) take(VBP) steps(NNS) forward(RB) every(DT) day(NN) no(DT) matter(NN) how(WRB)
small(JJ) they(PRP) may(MD) be(VB)
- (b) I solemnly swear that I am up to no good.
I(PRP) solemnly(RB) swear(VBP) that(IN) I(PRP) am(VBP) up(RB) to(IN) no(DT) good(NN)
- (c) Show what we truly are.
Show(VB) what(WP) we(PRP) truly(RB) are(VBP)
- (d) The pen is mightier than the sword.
The(DT) pen(NN) is(VBZ) mightier(JJR) than(IN) the(DT) sword(NN)
- (e) Live in the now.
Live(VB) in(IN) the(DT) now(NN)

While tagging the sentences, were there any particular challenges?

Some words can function as multiple parts of speech depending on context, like "good", "that" and "now".

Problem 3 (40 points)

We are given a Hidden Markov Model with the parameters below:

$S = \{N, M, V\}$ $K = \{\text{"Patrick"}, \text{"Cherry"}, \text{"can"}, \text{"will"}, \text{"see"}, \text{"spot"}\}$

π	
N	0.7
M	0.1
V	0.2

Table 2: Initial probabilities in our HMM model

	N	M	V
N	0.2	0.3	0.5
M	0.4	0.1	0.5
V	0.8	0.1	0.1

Table 3: Transition probabilities in our HMM model

	Patrick	Cherry	can	will	see	spot
N	0.3	0.2	0.1	0.1	0.1	0.2
M	0	0	0.4	0.6	0	0
V	0	0	0.1	0.2	0.5	0.2

Table 4: Emission probabilities in our HMM model

Answer the following questions.

- (a) Based on the model, how likely a sentence "Patrick can see Cherry" occur? Show your work.

$$N_{Patrick} = 0.7 \times 0.3 = 0.21$$

$$M_{Patrick} = 0.1 \times 0 = 0$$

$$V_{Patrick} = 0.2 \times 0 = 0$$

$$N_{can} = [N_1(0.2) + M_1(0.4) + V_1(0.8)]0.1 = 0.0042$$

$$M_{can} = [N_1(0.3) + M_1(0.1) + V_1(0.1)]0.4 = 0.0252$$

$$V_{can} = [N_1(0.5) + M_1(0.5) + V_1(0.1)]0.1 = 0.0105$$

$$N_{see} = [N_2(0.2) + M_2(0.4) + V_2(0.8)]0.1 = 0.001932$$

$$M_{see} = [N_2(0.3) + M_2(0.1) + V_2(0.1)]0 = 0$$

$$V_{see} = [N_2(0.5) + M_2(0.5) + V_2(0.1)]0.5 = 0.007875$$

$$N_{Cherry} = [N_3(0.2) + M_3(0.4) + V_3(0.8)]0.2 = 0.00133728$$

$$M_{Cherry} = [N_3(0.3) + M_3(0.1) + V_3(0.1)]0 = 0$$

$$V_{Cherry} = [N_3(0.5) + M_3(0.5) + V_3(0.1)]0 = 0$$

$$P = N_4 + M_4 + V_4 = 0.00133728$$

The probability of a sentence "Patrick can see Cherry" is 0.00133728.

- (b) Based on the model, how likely a sentence "will Cherry spot Patrick" occur? Show your work.

$$N_{will} = 0.7 \times 0.1 = 0.07$$

$$M_{will} = 0.1 \times 0.6 = 0.06$$

$$V_{will} = 0.2 \times 0.2 = 0.04$$

$$N_{Cherry} = [N_1(0.2) + M_1(0.4) + V_1(0.8)]0.2 = 0.014$$

$$M_{Cherry} = [N_1(0.3) + M_1(0.1) + V_1(0.1)]0 = 0$$

$$V_{Cherry} = [N_1(0.5) + M_1(0.5) + V_1(0.1)]0 = 0$$

$$\begin{aligned}
N_{spot} &= [N_2(0.2) + M_2(0.4) + V_2(0.8)]0.2 = 0.00056 \\
M_{spot} &= [N_2(0.3) + M_2(0.1) + V_2(0.1)]0 = 0 \\
V_{spot} &= [N_2(0.5) + M_2(0.5) + V_2(0.1)]0.2 = 0.0014 \\
N_{Patrick} &= [N_3(0.2) + M_3(0.4) + V_3(0.8)]0.3 = 0.0003696 \\
M_{Patrick} &= [N_3(0.3) + M_3(0.1) + V_3(0.1)]0 = 0 \\
V_{Patrick} &= [N_3(0.5) + M_3(0.5) + V_3(0.1)]0 = 0 \\
P &= N_4 + M_4 + V_4 = 0.0003696
\end{aligned}$$

The probability of a sentence "will Cherry spot Patrick" is 0.0003696.

- (c) Based on the model, find the most likely tag sequence of "Patrick can see Cherry". Show your work.

$$\begin{aligned}
N_{Patrick} &= 0.7 \times 0.3 = 0.21 \\
M_{Patrick} &= 0.1 \times 0 = 0 \\
V_{Patrick} &= 0.2 \times 0 = 0 \\
N_{can} &= \max[N_1(0.2), M_1(0.4), V_1(0.8)]0.1 = 0.0042 \\
M_{can} &= \max[N_1(0.3), M_1(0.1), V_1(0.1)]0.4 = 0.0252 \\
V_{can} &= \max[N_1(0.5), M_1(0.5), V_1(0.1)]0.1 = 0.0105 \\
N_{see} &= \max[N_2(0.2), M_2(0.4), V_2(0.8)]0.1 = 0.001008 \\
M_{see} &= \max[N_2(0.3), M_2(0.1), V_2(0.1)]0 = 0 \\
V_{see} &= \max[N_2(0.5), M_2(0.5), V_2(0.1)]0.5 = 0.0063 \\
N_{Cherry} &= \max[N_3(0.2), M_3(0.4), V_3(0.8)]0.2 = 0.001008 \\
M_{Cherry} &= \max[N_3(0.3), M_3(0.1), V_3(0.1)]0 = 0 \\
V_{Cherry} &= \max[N_3(0.5), M_3(0.5), V_3(0.1)]0 = 0
\end{aligned}$$

The most likely tag sequence of a sentence "Patrick can see Cherry" is NMVN.

- (d) Based on the model, find the most likely tag sequence of "will Cherry spot Patrick". Show your work.

$$\begin{aligned}
N_{will} &= 0.7 \times 0.1 = 0.07 \\
M_{will} &= 0.1 \times 0.6 = 0.06 \\
V_{will} &= 0.2 \times 0.2 = 0.04 \\
N_{Cherry} &= \max[N_1(0.2), M_1(0.4), V_1(0.8)]0.2 = 0.0064 \\
M_{Cherry} &= \max[N_1(0.3), M_1(0.1), V_1(0.1)]0 = 0 \\
V_{Cherry} &= \max[N_1(0.5), M_1(0.5), V_1(0.1)]0 = 0 \\
N_{spot} &= \max[N_2(0.2), M_2(0.4), V_2(0.8)]0.2 = 0.000256 \\
M_{spot} &= \max[N_2(0.3), M_2(0.1), V_2(0.1)]0 = 0 \\
V_{spot} &= \max[N_2(0.5), M_2(0.5), V_2(0.1)]0.2 = 0.00064 \\
N_{Patrick} &= \max[N_3(0.2), M_3(0.4), V_3(0.8)]0.3 = 0.0001536 \\
M_{Patrick} &= \max[N_3(0.3), M_3(0.1), V_3(0.1)]0 = 0 \\
V_{Patrick} &= \max[N_3(0.5), M_3(0.5), V_3(0.1)]0 = 0
\end{aligned}$$

The most likely tag sequence of a sentence "will Cherry spot Patrick" is VNVN.

Now, please implement the Viterbi algorithm from scratch in Python 3. Using any third party library or already existing implementation is not allowed. Using Numpy is fine.

Make a file named `run_viterbi.py` where the main function calls the function named `viterbi()`. Arguments of the `viterbi` will be all the HMM parameter values (i.e., initial probabilities, transition probabilities, and emission probabilities) and a test sentence.

In the main function, call the `viterbi` function twice, first to compute (c), and then to compute (d).

- (e) We will run your code using the command `python run_viterbi.py`. Write your code to print out mostly likely tag sequences for (c) and (d) with the probability for each state. You only need to print probabilities of the states in the most likely sequences. Print these to the screen.

```

• (env) jessiechen@JessiedeMacBook-Air-2 Downloads % /Users/jessiechen/Downloads/env/bin/python "/Users/jessiechen/Downloads/2026 Spring/NLP/HW/HW2/run_viterbi.py"
Part c: Patrick can see Cherry
Patrick  -> N (probability: 0.2100000000)
can      -> M (probability: 0.0252000000)
see      -> V (probability: 0.0063000000)
Cherry   -> N (probability: 0.0010080000)

Part d: will Cherry spot Patrick
will     -> V (probability: 0.0400000000)
Cherry   -> N (probability: 0.0064000000)
spot     -> V (probability: 0.0006400000)
Patrick  -> N (probability: 0.0001536000)

```

- (f) Are the most likely sequences you obtained by running your code the same as the sequences you obtained in (c) and (d)? If not, what would be the reason?

Yes, the sequences obtained from running the code match the manual calculations in parts (c) and (d).

Problem 4 (40 points)

We will build a logistic regression classifier, using a well-known machine learning python package `scikit-learn` (`sklearn`).

You will need to have `run_lr.py`. When executing the file using `python run_lr.py`, it needs to print your answers to (e) and (f) to the screen. (You need to include the answers in your pdf submission as well.)

First, download *polarity dataset v2.0* from this website. When uncompressing the file, you will find data in two folders: *pos* and *neg*. Yes, this is a dataset for sentiment analysis.

Sort the files in each folder in ascending order using the filenames. Move the last 200 samples in each folder to new folders (`pos_test`, `neg_test`), which will be our test data. Change the original folder names into `pos_train` and `neg_train`, respectively. Now your `pos_train` and `neg_train` have 800 samples each, and `pos_test`, `neg_test` have 200 samples each. The files in `_train` and `_test` folders must be different.

`pos` and `neg` will be the labels for the documents. When you read the documents in your code, your code will have to remember the label for each document.

We will only use adjectives as our features for the classifier.

Sklearn documentation

- (a) The documents are already tokenized, and separated by whitespace. Using the python NLP package, `spaCy` (documentation), obtain POS tags. Using the POS tags, make a vocabulary of only adjectives by using the training data.

```
# a
nlp = spacy.load('en_core_web_sm')

def get_adjectives(text, nlp_model):
    doc = nlp_model(text)
    return {token.text.lower() for token in doc if token.pos_ == 'ADJ'}

adj_vocab = set()
for text in train_docs:
    adj_vocab.update(get_adjectives(text, nlp))

adj_vocab = sorted(adj_vocab)
adj_index = {adj: i for i, adj in enumerate(adj_vocab)}
vocab_size = len(adj_vocab)
```

- (b) Now make bag-of-words binary features only for those adjectives. You will have a feature vector with the size of the adjective vocabulary. Then, each document can be represented as a feature vector – if an adjective appears in the document, you set the feature value 1, otherwise 0. Represent all the documents in training set and test set as feature vectors.

```
# b
def vectorize(docs, nlp_model, adj_index, vocab_size):
    X = np.zeros((len(docs), vocab_size), dtype=np.int8)
    for i, text in enumerate(docs):
        for adj in get_adjectives(text, nlp_model):
            if adj in adj_index:
                X[i, adj_index[adj]] = 1
    return X

X_train = vectorize(train_docs, nlp, adj_index, vocab_size)
y_train = np.array(train_labels)
X_test = vectorize(test_docs, nlp, adj_index, vocab_size)
y_test = np.array(test_labels)
```

- (c) Write code for logistic regression using sklearn (documentation. Use your own feature representations from (b) for training and testing. Train on the training data.

```
# c
clf = LogisticRegression(max_iter=1000, solver='lbfgs', random_state=42)
clf.fit(X_train, y_train)
```

- (d) Add code for evaluation metrics. We will use f1 score, and you can use `sklearn.metrics.f1_score` to obtain f1 score.

```
# d
train_f1 = f1_score(y_train, clf.predict(X_train))
test_f1 = f1_score(y_test, clf.predict(X_test))
```

- (e) Apply your trained model on the training data. What is the f1 score (i.e., training accuracy)?

```
(env) jessiechen@JessiedeMacBook-Air-2 HW2 % /Users/jessiechen/Downloads/env/bin/python "/Users/jessiechen/Downloads/2026 Spring/NLP/HW/HW2/run_lr.py"
(e) F1 Score on TRAINING data: 1.0000
(f) F1 Score on TEST data: 0.7797
```

- (f) Apply your trained model on the test data. What is the f1 score (i.e., test accuracy)?

```
(env) jessiechen@JessiedeMacBook-Air-2 HW2 % /Users/jessiechen/Downloads/env/bin/python "/Users/jessiechen/Downloads/2026 Spring/NLP/HW/HW2/run_lr.py"
(e) F1 Score on TRAINING data: 1.0000
(f) F1 Score on TEST data: 0.7797
```

- (g) Discuss the results. Were the features effective for this classification? In either case (yes or no), why do you think it was the case? Any additional features you can think of? Describe one idea. Yes, the features effective for this classification. The gap between training F1 (1.0) and test F1 (0.78) suggests the model may be overfitting.

Adjectives alone are limited because negation is ignored ("not good" is treated the same as "good"), and other sentiment-bearing words like verbs ("love", "hate") and nouns ("masterpiece", "disaster") are excluded.

One additional feature idea: negation bigrams — treating "not + adjective" as a single feature (e.g., not good vs. good), since negation flips sentiment polarity.